第 16 节 · 加餐：V4 项目 Docker 部署

至此，你已经搞定:

```
你的笔记本
  ├── OpenClaw daemon 跑着        ← 笔记本一开就活
  ├── cron 每天 8:00 自动推送      ← 笔记本不关推送就来
  └── Telegram Bot 互动检索        ← 笔记本醒着 Bot 就在
```

Bot 现在依附于你的笔记本活着，笔记本关机 = Bot 断线 = 推送停。

选项对比

你的目标	要不要做 16-1	该走哪条路
自己用够了,关机一会儿无所谓	❌ 不用	已达成，直接 实操 2 毕业
采集要 7×24 不掉线(我睡觉也得采)	❌ 不用	**GitHub Actions**(`daily-collect-v4.yml` 已配,每天 UTC 00:00 自动跑,不依赖你笔记本)
Bot 也要 7×24 在线(半夜有人问也答)	✅ 必做	云服务器 + Docker
要 share 给别人用 / 想做产品	✅ 必做	Docker 镜像 + 上线 Checklist

Docker 到底做了什么?

Docker 不是让你笔记本上更好用。

Docker 是为了离开你的笔记本，把整套 v4 系统装进标准盒子，搬到任何地方都能跑。

- 一台云服务器(阿里云 / Hetzner,月租 30-100 元)
- 家里的 NAS / 树莓派
- 给同事 share，他装个 Docker 就能跑你的 Bot

核心价值一句话：**环境一致性 + 7×24 在线。**

对**大部分朋友来说**:

```
笔记本本机 (15 节状态)  +  GitHub Actions 自动采集  =  已经够用
                                                        ↑
                                                  你已经达到这里
```

GitHub Actions 替你解决了“采集 7×24”，你的笔记本关着也没事。Bot 互动只发生在笔记本醒着的时段，对学习/试用**完全够**。

Docker（本节）是给/；想做产品 / 上线 SaaS”的同学提供的加餐，这是工程化进阶，**不是毕业必做。**

目的

不是“必须把 Bot 装进容器”，而是教你 3 件事。

1. **理解部署边界**： 哪些组件能进容器（pipeline / bot）、哪些不能（OpenClaw daemon 不进容器）
2. **理解环境一致性** ：“在我电脑能跑”不等于“在你电脑能跑”
3. **理解 Checklist 文化**：上线前要查的 10 项验证 Checklist 其实比 Docker 本身更重要）

目标：Docker 装好 + 镜像构建成功 + bot 容器常驻 + pipeline 容器手动触发能跑通
目标文件：`Dockerfile` + `docker-compose.yml` + `pipeline/pipeline.py`（薄封装改写）+ host cron

注意，Docker安装步骤多，各种环境中的情况也不同，而且需要很多前置知识。因此我们很难有一个清晰一致的步骤清单，下面的步骤只是大概参考。具体细节需要大量的和Claude Code等AI工具切磋，或者网上查阅资料，或者群里面探讨。

步骤 0：前置确认（v4-production 已就绪）

如果你跑过 13-2 步骤 1：v4-production 已经从 v3 cp 过来了（含 workflows / patterns / pipeline / hooks / tests / knowledge / .opencode / AGENTS.md）+ 13-1 onboard 生成的 openclaw/。Docker 加餐**只需要补一个空目录**给运行时日志：

```
cd ~/ai-knowledge-base/v4-production
mkdir -p data
```

如果你直接来跑 Docker 加餐没做过 13-2：用下面的最小起步：

```
cd ~/ai-knowledge-base
mkdir -p v4-production && cd v4-production
cp -r ../v3-multi-agent/{workflows,patterns,pipeline,hooks,tests,knowledge,.opencode,AGENTS.md} .
mkdir -p distribution bot openclaw/skills scripts data
touch distribution/__init__.py bot/__init__.py
```

期望目录：

```
v4-production/
├── workflows/        ← 继承 V3
├── patterns/         ← 继承 V3
├── pipeline/         ← 继承 V3 · 步骤 2 改写为薄封装
├── distribution/     ← 14 节填实
├── bot/              ← 15 节填实
├── openclaw/         ← 13 节生成的网关配置
```

```
├── data/              ← 运行时日志（本节新建）
└── knowledge/         ← 知识库（v3 cp 来）
```

步骤 1：装 Docker（Ubuntu 22.04+）

> **跳过条件**：`docker --version && docker compose version` 都有输出且非 root 能跑就跳到步骤 2。

1.1 卸载老版本（如果有）

```
for pkg in docker.io docker-doc docker-compose docker-compose-v2 podman-
    sudo apt-get remove -y $pkg 2>/dev/null
done
```

1.2 装 Docker CE 官方版

```
# 准备 apt 源
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt
sudo chmod a+r /etc/apt/keyrings/docker.asc

# 加 Docker 官方仓库
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings
    $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 装
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-bui
```

1.3 让当前用户免 sudo 跑 docker

```
sudo usermod -aG docker $USER
newgrp docker        # 当前 shell 立刻生效（或重新登陆 shell）
```

1.4 验证

```
docker --version           # Docker version 27.x.x
docker compose version     # Docker Compose version v2.x.x
docker run --rm hello-world
```

看到 `Hello from Docker!` 就成。

步骤 2：改写 pipeline/pipeline.py 为 V3 LangGraph 薄封装

V4 的 pipeline 不重写采集逻辑 —— 直接调 V3 LangGraph workflow，加上分发：

```
"""pipeline/pipeline.py — V4 一次完整执行入口（被 cron 触发）"""

import logging
import sys
from pathlib import Path

from workflows.graph import app as v3_workflow      # V3 LangGraph 核心
from distribution.publisher import publish_daily_digest

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname
log = logging.getLogger(__name__)


def run_once() -> int:
    """跑一次完整流水线：V3 工作流采集分析 → 分发推送。返回退出码。"""
    log.info("=== V4 pipeline 启动 ===")

    # 1. V3 LangGraph：采集 + 分析 + 审核 + 入库
    initial_state = {
        "sources": [], "analyses": [], "articles": [],
        "review_feedback": "", "review_passed": False,
        "iteration": 0, "needs_human_review": False,
        "plan": {}, "cost_tracker": {},
    }
    final_state = v3_workflow.invoke(initial_state)
    log.info(f"V3 完成：{len(final_state.get('articles', []))} 条新条目")

    # 2. 分发（异步推送到 Telegram / 飞书）
    import asyncio
    results = asyncio.run(publish_daily_digest())
    for r in results:
        log.info(f"  {r.channel}: {'✓' if r.success else '✗'} {r.message

    return 0 if all(r.success for r in results) else 1


if __name__ == "__main__":
    sys.exit(run_once())
```

关键设计：`pipeline.py` 是**纯一次性脚本**（run_once 跑完退出），不是常驻进程。这样它适合 cron 调度 —— host cron 或 docker compose run 都能拉起一次就退出。

步骤 3：写 Dockerfile（多阶段构建）

```
# ============== 构建层 ==============
FROM python:3.12-slim AS builder
```

```
WORKDIR /build
RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc && \
    rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ============== 运行层 ==============
FROM python:3.12-slim
LABEL version="4.0"

WORKDIR /app
COPY --from=builder /install /usr/local

# 先 V3 继承（workflows / patterns），再 V4 新增（pipeline / distribution /
# 顺序体现 "V4 = V3 + 分发 + Bot"
COPY workflows/ ./workflows/
COPY patterns/ ./patterns/
COPY pipeline/ ./pipeline/
COPY distribution/ ./distribution/
COPY bot/ ./bot/

# 运行时数据目录（volume 会挂这里）
RUN mkdir -p /app/knowledge/articles /app/knowledge/raw /app/data

# 非 root 用户跑
RUN groupadd -r appuser && useradd -r -g appuser appuser && \
    chown -R appuser:appuser /app
USER appuser

# 健康检查 · 验证模块能 import（最低限度的"代码完整性"探针）
HEALTHCHECK --interval=60s --timeout=10s --start-period=30s --retries=3
    CMD python -c "import bot.knowledge_bot" || exit 1

CMD ["python", "-m", "bot.knowledge_bot"]
```

这版 Dockerfile 的关键决策：

决策	理由
多阶段构建	builder 装 gcc 编译 wheel，运行层不带编译器 → 镜像瘦
python:3.12–slim	Debian slim 体积约 50MB · 加 Python 后 ~150MB
非 root appuser	容器逃逸时降低危害面
不装 cron	容器单一职责 —— pipeline 用 host cron 触发，bot 长驻自己跑
HEALTHCHECK 用 import bot.knowledge_bot	至少验证模块加载链没断（比 sys.exit(0) 真实）

> `requirements.txt` **没有？** `cd ~/ai-knowledge-base/v4-production && pip freeze | grep -E "^(langgraph|openai|aiohttp|python-dotenv)" > requirements.txt`，挑你真用到的，别 pip freeze 全量（会带一堆开发工具）。

步骤 4：写 docker-compose.yml

设计要点：

- **bot 长驻**（`compose up` 起来，挂掉自动重启）
- **pipeline 一次性**（不在默认启动里 · `docker compose run pipeline` 手动触发，host cron 调）
- **bind mount knowledge/ + data/**（让宿主机 OpenClaw daemon 能直接读同一份 knowledge/，不要用 named volume 隔开）

```yaml
services:
  bot:
    build: .
    image: kb-v4:latest
    container_name: kb-bot
    restart: unless-stopped
    command: ["python", "-m", "bot.knowledge_bot"]
    volumes:
      - ./knowledge:/app/knowledge
      - ./data:/app/data
    env_file: [.env]
    environment:
      - TZ=Asia/Singapore
      - PYTHONPATH=/app
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"

```

```
  pipeline:
    build: .
    image: kb-v4:latest
    container_name: kb-pipeline
    profiles: ["manual"]    # 关键 · compose up 不会启动这个 · 仅 docker co
    command: ["python", "-m", "pipeline.pipeline"]
    volumes:
      - ./knowledge:/app/knowledge
      - ./data:/app/data
    env_file: [.env]
    environment:
      - TZ=Asia/Singapore
      - PYTHONPATH=/app

# OpenClaw 网关不进 compose
# OpenClaw 是 npm 装的 Node CLI，跑在宿主机：
#   openclaw daemon start
# 它通过 ./knowledge 目录跟容器共享数据
```

`profiles: ["manual"]` **是核心**，它告诉 compose“默认不启动 pipeline，要跑就显式拉起来”。这样：

```
docker compose up -d            # 只起 bot · pipeline 不动
docker compose run --rm pipeline   # 触发一次 pipeline · 跑完退出
```

host cron 触发 pipeline

```
crontab -e
```

加两行（每天 8:00 / 20:00 跑一次）：

```
0 8,20 * * * cd /home/$USER/ai-knowledge-base/v4-production && /usr/bin/
```

这是**最干净的 Docker 调度模式** —— 把“什么时候跑”的问题留给 host cron（host 上的 cron 已经够稳），容器只负责“怎么跑”。

步骤 5：确认 .env 和 .gitignore

```
cd ~/ai-knowledge-base/v4-production

# .env 应有 LLM_API_KEY / TELEGRAM_BOT_TOKEN / TELEGRAM_CHAT_ID（13-1 / 1
cat .env | grep -E "LLM_API_KEY|TELEGRAM_BOT_TOKEN" | sed 's/=.*/=***mas

# 确认 .env 在 .gitignore
grep -q "^\.env$" .gitignore || echo ".env" >> .gitignore

# 收紧 .env 权限
chmod 600 .env
```

步骤 6：构建镜像

```
cd ~/ai-knowledge-base/v4-production
docker compose build
docker images | grep kb-v4
```

期望：镜像约 200-280 MB（Python 3.12-slim 基础 + 你 requirements 的依赖体积决定）。

常见问题：

问题	解决方案
requirements.txt 缺失	`pip freeze > requirements.txt`
npm 报错	Dockerfile 中 `npm ci ... || true` 已处理
权限问题	确保 Docker Desktop 已启动

步骤 7：启动 bot 容器

```
docker compose up -d          # -d 后台跑
docker compose ps             # 看 bot 状态
docker compose logs -f bot    # 看 bot 实时日志（Ctrl+C 退出 follow）
```

期望 `docker compose ps` 输出含 `kb-bot · running`。

步骤 8：测试健康检查

```
# 等 30 秒（HEALTHCHECK 的 start-period）
sleep 35
docker inspect --format='{{.State.Health.Status}}' kb-bot
# 期望: healthy
```

如果是 `unhealthy`，看为什么模块加载失败：

```
docker exec kb-bot python -c "import bot.knowledge_bot"
# 报错信息直接告诉你哪个 import 不通
```

步骤 9：手动触发 pipeline（验证一次性容器）

```
docker compose run --rm pipeline
```

期望输出：

```
2026-... [INFO] === V4 pipeline 启动 ===
2026-... [INFO] V3 完成：N 条新条目
2026-... [INFO]   telegram: ✓ <message_id>
```

跑完容器自动 `--rm` 删除（一次性）。host cron 后续会按 8:00 / 20:00 自动触发。

步骤 10：测试自动重启

```
# 故意杀掉 bot 进程
docker exec kb-bot pkill -9 python || true

# 等 10 秒看 restart 起没起来
sleep 10
docker compose ps
# 期望: kb-bot 又 running 了 · uptime 重置
```

`restart: unless-stopped` 保证 bot 挂了自动拉起来 —— 这是生产环境的最低要求。

步骤 11：停止并验证数据持久化

```
docker compose down       # 停 + 删容器（不删 image，不删 bind mount 数据）
ls knowledge/articles/    # bind mount · 数据还在
ls data/                  # 同上
docker compose up -d      # 重新起 · 数据没丢
```

bind mount 的好处，**容器即使删了重建，你的知识库不会丢。**

步骤 12：打版本标签

```
docker tag kb-v4:latest kb-v4:$(date +%Y%m%d)
docker images | grep kb-v4
# kb-v4   latest      <hash>   ...
# kb-v4   20260502    <hash>   ...
```

带日期 tag 是上线时回滚的关键 —— `docker compose down && docker tag kb-v4:20260501 kb-v4:latest && docker compose up -d` 一键回到昨天的版本。

步骤 13：提交到 Git

```
git add Dockerfile docker-compose.yml pipeline/pipeline.py requirements.
git commit -m "feat: add docker deployment with split bot/pipeline conta
```

注意：`.env` 必须在 `.gitignore` 里，**绝不能 commit token。**

扩展（可选）

- **加飞书 webhook 推送**：`distribution/publisher.py` 已经有 `FeishuPublisher`，给 `.env` 加 `FEISHU_WEBHOOK_URL=...`，pipeline 跑完就两个渠道一起推。
- **改 Postgres 持久化 cost-report**：当前 `data/cost-report.json` 是文件。生产环境想多容器汇总 token 消耗，加一个 postgres 服务到 compose，pipeline 写 DB。

- **加 Prometheus + Grafana 监控**：bot / pipeline 的 logging 改 structured log（json），写到 file。再加个 promtail + Loki + Grafana stack 看面板。这是真生产的 observability 起点。
- **多环境（dev/staging/prod）**：拆 `docker-compose.yml` + `docker-compose.prod.yml` 用 `-f` 叠加。dev 用 `image: kb-v4:dev`，prod 用 `kb-v4:20260502` 显式版本。

Docker 不是把笔记本上的事抽象一遍，是为了“离开笔记本”。pipeline 用 host cron 触发的 one-shot 容器、bot 用 long-lived 的 always-on 容器、OpenClaw 留在 host —— 三个组件各按自己的生命周期跑，bind mount 让它们共享同一份知识库。这就是真正的“分布式应用最小构型”。